Blog Starter Template - Extracted from Tarot Pathwork

🎯 What This Template Includes

A production-ready blog system with:

- **Tiptap Rich Text Editor** with image upload
- **Supabase Backend** for auth & database
- **Admin Dashboard** with post management
- **Public Blog** with pagination & SEO
- **Analytics Integration** (Google Analytics + custom tracking)
- **Social Sharing** built-in
- **Responsive Design** (shadcn/ui + Tailwind)

📦 Core Components to Extract

1. Blog Editor & Management

```
src/components/RichTextEditor.tsx     # The Tiptap editor component
src/pages/CreatePostPage.tsx          # Create new posts
src/pages/EditPostPage.tsx            # Edit existing posts
src/pages/PostsListPage.tsx           # Admin post list
src/components/AdminLayout.tsx        # Admin sidebar navigation
src/pages/AdminNewPost.tsx            # Login page
src/pages/AuthCallback.tsx            # OAuth callback handler
```

2. Public-Facing Blog

```
src/pages/BlogIndexPublic.tsx         # Main blog listing with pagination
src/pages/BlogPost.tsx                # Individual post view with sharing
src/components/SEO.tsx                # SEO meta tags component
```

3. Supporting Infrastructure

```
src/lib/supabase.ts              # Supabase client setup
src/lib/PostAnalytics.ts           # View tracking system
api/cron/sync-analytics.ts          # GA4 sync (optional)
```

4. UI Components (from shadcn/ui)

```
src/components/ui/button.tsx
src/components/ui/card.tsx
src/components/ui/input.tsx
src/components/ui/label.tsx
src/components/ui/alert-dialog.tsx
src/hooks/use-toast.tsx
```

🗄️ Database Schema

Required Supabase Tables

`posts` table

```sql
```

```sql
CREATE TABLE public.posts (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  title text NOT NULL,
  content text NOT NULL,
  author_id uuid NOT NULL REFERENCES auth.users(id),
  published boolean DEFAULT false,
  slug text UNIQUE,
  excerpt text,
  cover_image text,
  created_at timestamptz DEFAULT now(),
  updated_at timestamptz DEFAULT now(),
  published_at timestamptz,
  tags text[] DEFAULT '{}',
  view_count integer DEFAULT 0,
  like_count integer DEFAULT 0,
  meta_description text,
  meta_keywords text[],
  last_ga_sync timestamptz
);

-- Indexes for performance
CREATE INDEX idx_posts_published ON posts(published);
CREATE INDEX idx_posts_published_at ON posts(published_at DESC);
CREATE INDEX idx_posts_author ON posts(author_id);
CREATE INDEX idx_posts_slug ON posts(slug);
```

`post_views` table (for analytics)

sql

```sql
CREATE TABLE public.post_views (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  post_id uuid NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  viewed_at timestamptz DEFAULT now(),
  user_id uuid REFERENCES auth.users(id),
  ip_address text
);

CREATE INDEX idx_post_views_post_id ON post_views(post_id);
CREATE INDEX idx_post_views_viewed_at ON post_views(viewed_at DESC);
```

Row Level Security (RLS) Policies

sql

```sql
-- Enable RLS
ALTER TABLE posts ENABLE ROW LEVEL SECURITY;
ALTER TABLE post_views ENABLE ROW LEVEL SECURITY;

-- Posts: Anyone can read published posts
CREATE POLICY "Public posts are viewable by everyone"
  ON posts FOR SELECT
  USING (published = true);

-- Posts: Authors can read their own posts (including drafts)
CREATE POLICY "Authors can view their own posts"
  ON posts FOR SELECT
  USING (auth.uid() = author_id);

-- Posts: Authors can create posts
CREATE POLICY "Authors can create posts"
  ON posts FOR INSERT
  WITH CHECK (auth.uid() = author_id);

-- Posts: Authors can update their own posts
CREATE POLICY "Authors can update their own posts"
  ON posts FOR UPDATE
  USING (auth.uid() = author_id);

-- Posts: Authors can delete their own posts
CREATE POLICY "Authors can delete their own posts"
  ON posts FOR DELETE
  USING (auth.uid() = author_id);

-- Post Views: Anyone can insert views
CREATE POLICY "Anyone can track post views"
  ON post_views FOR INSERT
  WITH CHECK (true);

-- Post Views: Anyone can read views
CREATE POLICY "Views are publicly readable"
  ON post_views FOR SELECT
  USING (true);
```

⚙️ Configuration Files

1. Environment Variables (`.env`)

bash

```bash
# Supabase
VITE_SUPABASE_URL=https://your-project.supabase.co
VITE_SUPABASE_ANON_KEY=your-anon-key-here

# For server-side operations (Vercel functions)
SUPABASE_SERVICE_ROLE_KEY=your-service-key-here

# Google Analytics (optional)
GA4_PROPERTY_ID=your-ga4-property-id
GA4_CREDENTIALS_JSON={"type":"service_account",...}
```

2. Supabase Storage Bucket

For image uploads, create a public bucket:

sql

```sql
-- In Supabase Storage, create bucket "blog-images"
-- Set as public
-- Add CORS policy for your domain
```

Update RichTextEditor.tsx to use your bucket:

typescript

```typescript
const { data, error } = await supabase.storage
  .from('blog-images')  // Your bucket name
  .upload(fileName, file);
```

🚀 Setup Instructions

Step 1: Initialize New Project

bash

```bash
npm create vite@latest my-blog -- --template react-ts
cd my-blog
npm install
```

Step 2: Install Dependencies

bash

```bash
# Core dependencies
npm install @supabase/supabase-js
npm install react-router-dom
npm install lucide-react
npm install date-fns

# Tiptap editor
npm install @tiptap/react @tiptap/starter-kit
npm install @tiptap/extension-image
npm install @tiptap/extension-link
npm install @tiptap/extension-underline
npm install @tiptap/extension-text-align
npm install @tiptap/extension-placeholder

# UI components (shadcn/ui)
npm install class-variance-authority clsx tailwind-merge
npm install @radix-ui/react-dialog
npm install @radix-ui/react-alert-dialog
npm install @radix-ui/react-toast

# Tailwind CSS
npm install -D tailwindcss postcss autoprefixer
npx tailwindcss init -p
```

Step 3: Copy Core Files

1. Copy all files from the "Core Components" section above

2. Copy UI components from `src/components/ui/`
3. Copy utility files from `src/lib/`

Step 4: Configure Routing

Update `src/main.tsx`:

typescript

```typescript
import { createBrowserRouter, RouterProvider } from 'react-router-dom';
import Layout from './components/Layout';
import AdminLayout from './components/AdminLayout';
import BlogIndex from './pages/BlogIndexPublic';
import BlogPost from './pages/BlogPost';
import CreatePostPage from './pages/CreatePostPage';
import EditPostPage from './pages/EditPostPage';
import PostsListPage from './pages/PostsListPage';
import AdminNewPost from './pages/AdminNewPost';
import AuthCallback from './pages/AuthCallback';

const router = createBrowserRouter([
  {
    path: '/',
    element: <Layout />,
    children: [
      { path: 'blog', element: <BlogIndex /> },
      { path: 'blog/:slug', element: <BlogPost /> },
      { path: 'admin/login', element: <AdminNewPost /> },
      { path: 'auth/callback', element: <AuthCallback /> },
    ],
  },
  {
    path: '/admin',
    element: <AdminLayout />,
    children: [
      { path: 'posts', element: <PostsListPage /> },
      { path: 'posts/new', element: <CreatePostPage /> },
      { path: 'posts/:id/edit', element: <EditPostPage /> },
    ],
  },
]);
```

Step 5: Set Up Supabase

1. Create project at https://supabase.com
2. Run the SQL schema from the "Database Schema" section
3. Enable Google OAuth (Settings → Authentication → Providers)
4. Create storage bucket for images
5. Copy environment variables to `.env`

Step 6: Customize Branding

Update these files to match your brand:

Tailwind Config (`tailwind.config.js`):

javascript

```javascript
theme: {
  extend: {
    colors: {
      // Replace tpblue, tpgold, etc. with your colors
      primary: {
        DEFAULT: '#your-primary-color',
        foreground: '#your-text-color',
      },
      // ... more colors
    },
  },
}
```

Layout Component (`src/components/Layout.tsx`):

- Update navigation links
- Replace logo/branding
- Customize footer

🎨 Customization Checklist

Must Change

- [ ] Brand colors in `tailwind.config.js`
- [ ] Site name in `Layout.tsx` and `SEO.tsx`
- [ ] Domain in `VITE_SITE_URL` environment variable
- [ ] Supabase project credentials
- [ ] Storage bucket name for images
- [ ] Google Analytics ID (if using)

Optional

- [ ] Font families (currently uses Inter)
- [ ] Add/remove editor toolbar buttons
- [ ] Customize SEO defaults
- [ ] Add categories/tags UI
- [ ] Add comments system
- [ ] Add newsletter integration

📝 Key Features to Configure

Image Upload

The RichTextEditor handles image uploads automatically. To customize:

typescript

```typescript
// In RichTextEditor.tsx
const handleImageUpload = async (file: File) => {
  const fileName = `${Date.now()}-${file.name}`;

  const { data, error } = await supabase.storage
    .from('YOUR-BUCKET-NAME')  // Change this
    .upload(fileName, file);

  // Returns public URL
  return `${supabaseUrl}/storage/v1/object/public/YOUR-BUCKET-NAME/${fileName}`;
};
```

SEO Configuration

Default SEO values in `src/components/SEO.tsx`:

typescript

```typescript
const defaults = {
  siteName: 'Your Blog Name',
  twitterHandle: '@yourhandle',
  defaultImage: '/og-image.jpg',
};
```

Social Sharing

BlogPost.tsx includes sharing for:

- Twitter
- Facebook
- LinkedIn
- Copy link

Update the share URLs if you need different platforms.

🔧 Deployment

Vercel (Recommended)

bash

```bash
npm install -g vercel
vercel
```

Add environment variables in Vercel dashboard.

Other Platforms

Works with any static hosting:

- Netlify
- Cloudflare Pages
- AWS Amplify
- Traditional web hosts

Build command: `npm run build` Output directory: `dist`

📊 Analytics Setup (Optional)

Google Analytics 4

1. Create GA4 property
2. Add GA4 Measurement ID to `.env`
3. Optional: Set up server-side sync with `api/cron/sync-analytics.ts`

Custom View Tracking

The `PostAnalytics.ts` library tracks views:

- Deduplicates by IP + localStorage
- 24-hour cooldown per user
- Stores in `post_views` table

To disable: Remove `trackPostView()` call from `BlogPost.tsx`

🛠️ Common Modifications

Add Categories

sql

```sql
CREATE TABLE categories (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL,
  slug text UNIQUE NOT NULL
);

ALTER TABLE posts ADD COLUMN category_id uuid REFERENCES categories(id);
```

Add Author Profiles

sql

```sql
CREATE TABLE author_profiles (
  user_id uuid PRIMARY KEY REFERENCES auth.users(id),
  display_name text,
  bio text,
  avatar_url text
);
```

Add Comments

Use Supabase Realtime or integrate:

- Disqus
- Utterances (GitHub issues)
- Custom solution with `comments` table

🐛 Troubleshooting

Images not uploading

- Check storage bucket is public
- Verify CORS settings in Supabase
- Check file size limits

Can't create posts

- Verify RLS policies are set correctly
- Check user is authenticated
- Ensure `author_id` matches authenticated user

Analytics not working

- Verify GA4 credentials
- Check cron job is running
- Review browser console for errors

📚 Tech Stack Reference

Feature	Library	Version
Framework	React + Vite	Latest
Database	Supabase	Latest
Editor	Tiptap	3.10+
Routing	React Router	7.x
UI	shadcn/ui	Latest
Styling	Tailwind CSS	3.x

🎓 Learning Resources

- Tiptap Docs
- Supabase Docs
- shadcn/ui Components
- React Router

📄 License Recommendation

Add a `LICENSE` file to your project. MIT License is recommended for templates:

```
MIT License

Copyright (c) 2024 Your Name

Permission is hereby granted, free of charge, to any person obtaining a copy...
```

✅ Production Checklist

Before launching:

- [ ] Set up proper error tracking (Sentry, etc.)

- ☐ Configure backup strategy for Supabase
- ☐ Set up SSL certificate (auto with Vercel/Netlify)
- ☐ Test all forms with validation
- ☐ Verify SEO meta tags with testing tools
- ☐ Check mobile responsiveness
- ☐ Test authentication flow
- ☐ Review RLS policies for security
- ☐ Set up monitoring/uptime alerts
- ☐ Configure proper CORS headers
- ☐ Add rate limiting for API routes
- ☐ Set up staging environment

🚀 What's NOT Included

These features from Tarot Pathwork are **not** part of the blog starter:

- Astronomy/astrology calculations
- Tree of Life visualizations
- Booking system
- Payment processing (Stripe)
- Google Calendar integration
- Natal chart features
- Kabbalah-specific content
- Shem Angels calculator
- Resources/downloads system (but easily adaptable)

However, the booking/payment components could be extracted separately if needed.

Need help? Review the original files for implementation details. All components are well-commented and follow React best practices.